Understanding driver engagement for a drive sharing App

Objective: To transform raw data into a data product (Drivers Activity) that will be consumed by different data teams in the company, to measure driver engagement.

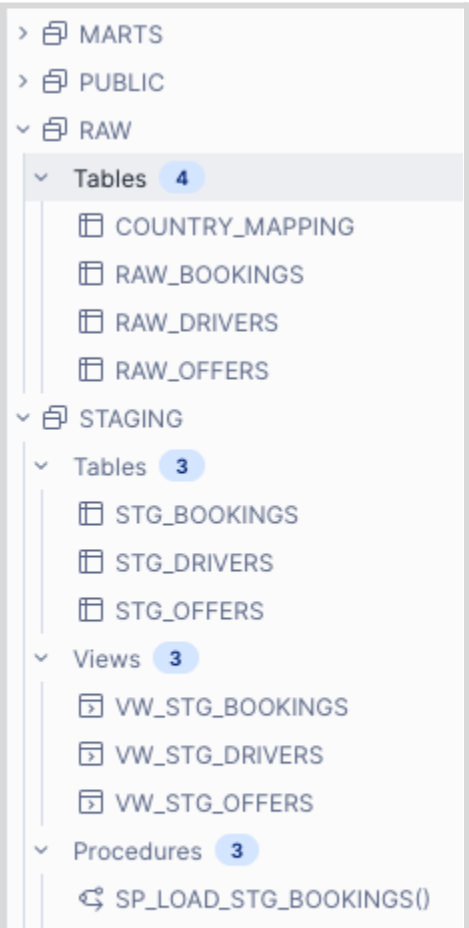
Input Files:

1. drivers.csv - Snapshot of the last information of each driver
   id | Date_registration | Driver_rating | Rating_count | Receive_marketing | Country

2. offers.csv - One entry per offer sent to the drivers
   id | datecreated | bookingid | driverid | routedistance | state (ACCEPTED or CANCELED | driverread
3. bookings.csv - One entry per booking created by the passengers
   id | request_date | status | id_driver | Estimated_route_fare

1. Snowflake Setup & Initial load

A trial snowflake account was created. The setup.sql script was executed to create role, warehouse, database, schema and tables.



2. Schema Design Overview

raw schema:
All source data is preserved in its original form. Columns are stored as VARCHAR to avoid transformation at ingestion time and ensure data immutability. This allows the pipeline to be rebuilt from raw if any downstream issue occurs.

staging schema:
Contains deduplicated and standardized records derived from raw. Data types are converted appropriately, null handling is applied, and data quality checks are performed. No records are deleted; instead, invalid records are flagged using an is_valid indicator to maintain traceability.

marts schema:

Stores curated fact tables, dimension tables, and aggregate models. This is the business-ready layer used for reporting and analytical consumption.

analyses schema:
Includes analytical views and derived tables created for KPI exploration and trend analysis. These are built using combinations of aggregates, facts, and dimensions.

utils schema:
Contains internal stages and supporting objects used for raw data ingestion and pipeline operations.

3. Initial Data Load to Raw schema

- **Compression**: CSV files are compressed to .gz format to save on upload time and Snowflake storage costs (*gzip -c offers.csv > offers.csv.gz*)
- **Stage Setup**: Created internal stages in Snowflake (e.g., int_offers_stage) to act as the landing zone for the files.
- **Ingestion (Snow CLI)**: Used the SnowSQL/CLI PUT command to move local files into the Snowflake stage:
  PUT file:///Users/devikah/Downloads/AE_Case_Study/offers.csv.gz @int_offers_stage;
- **Load to Raw:** Populated the raw tables using the COPY INTO command, which handles the bulk ingestion from the stage.

  Additional added fields in raw:

  - **is_processed** - A Processing Flag used for Incremental loading to staging. This ensures the pipeline only transforms "new" data, significantly reducing Snowflake compute costs.[Default: False]
  - **load_tz** - Records the exact time data was ingested into the raw Layer. This provides a critical Audit Trail and allows us to track Data Recency and batch identification for troubleshooting.[Default: current_timestamp()]

4. Mapping Tables (setup.sql)

country_mapping: Since the source data is messy and has country names in different formats (e.g., "AT", "ÖSTERREICH", "Austria."), this mapping table is built to standardize everything into a single format. Each time a new country is added, this table needs to be updated.

5. Staging & Data Quality Checks

Before implementing data quality checks, the project objective — analyzing driver engagement — was revisited to ensure validations were aligned with business goals.

Following exploratory analysis (identifying null values, duplicate records, and relationships between tables), the cleaned data is moved into the staging layer. At this stage, columns are mapped to their appropriate data types, and structural inconsistencies are addressed.

Across the project, all date and timestamp fields are standardized using the TIMESTAMP_TZ format to ensure consistent timezone handling and accurate time-based analysis.

Data Quality columns added:

- **Is_valid** -> A boolean flag to quickly filter out bad data
- **Error_reason** -> A text field that captures exactly why a record failed (e.g., "Missing Driver ID")- for debugging.

Additional columns:

- raw_load_tz - the load timestamp when the data entered the data warehouse is carried forward to this layer to troubleshoot and trace any record back to its specific ingestion batch.
- Is_processed flag - used to manage the incremental flow so the same data is not transformed twice.

Naming convention: Column names follow a standardized entity_attribute format.
Example: driver_id is used consistently across all tables (instead of variations like id_driver or driverid). This ensures clarity, join consistency, and improved maintainability.

Every table has a **view** with only **valid records** from where the next layer would read from.
All error records are stored in staging itself for the team to review later.
These are the data quality checks:

Drivers

Column	Key Checks
ID	Not null
date_registration	Not null; > 2009-06-01(Freenow founding date) and ≤ current date
rating	Between 1–5; must exist if rating_count > 0

rating_count	Positive if rating exists
receive_marketing	Accepted values ('TRUE','FALSE','1','0','YES','NO')
country	Not null; exist in country_mapping

Bookings

Column	Key Checks
id	Not null
request_date	Not null; > 2009-06-01 and ≤ current date
status	Not null; Accepted values (SERVER_ABORT, SUCCESS, DRIVER_ABORT, NO_DRIVER_FOUND, PASSENGER_ABORT)
id_driver	Flagged if missing in drivers table (late arriving?)
estimated_route_fare	≥ 0

The "Ghost Driver" Issue
17% of booking records have driver_ids that don't exist in our driver table.

Flagging vs. Deleting: As this volume is so high, chose to flag them with **is_missing_driver** = TRUE instead of deleting them. This keeps our "Total Demand" numbers accurate while warning downstream users that driver-specific metrics for these rows have issues.

Handling NULLs: Allowed NULL values for driver_id as passengers often book and then cancel/abort before a driver is even assigned. These are still valid business events.

Offers

Column	Key Checks
id	Not null
created_date	Not null; > 2009-06-01 and ≤ current date
booking_id	Not null; flagged if missing in bookings
driver_id	Not null; flagged if missing in drivers

route_distance	> 0 (Location of client)
offer_state	Not null; Accepted values ('ACCEPTED','CANCELED')
driver_read	Not null; Accepted values ('TRUE','FALSE','1','0','YES','NO')

The "Ghost Driver" Issue

Similar to bookings, ~21% offers are generated to ghost drivers (4425) and these are flagged as is_missing_drivers. A small percentage of records have missing booking_ids also, which are flagged as **is_missing_booking**.

Advance Booking

It is possible for an offer related to a booking to be generated on a later date in the case of advance bookings.

Pipeline Execution

Scripts: [stg_drivers.sql -> stg_bookings.sql -> stg_offers.sql]

- **Dependency:** The scripts are executed in a specific sequence: stg_drivers.sql -> stg_bookings.sql -> stg_offers.sql. This order is mandatory to ensure the driver dimension is populated before the transaction tables try to reference them.
- **Stored Procedures:** Each staging load is wrapped in a Stored Procedure. This keeps the logic modular and allows us to trigger the entire load with a single command.
- **Transactional Integrity:** Both the data migration to Staging and the is_processed flag update in Raw are handled within a single **BEGIN — END block**.
    - This ensures the process is **Atomic**: if any part of the load fails, the whole transaction rolls back.
    - We don't want to mark records as "processed" if they didn't actually make it into the staging table. This setup prevents data gaps and ensures idempotency.

Data Quality Metrics

Table	Total Records	Valid Records	Driver #	Bookings #	Error records	Key Integrity Issue
drivers	6481	6196	6196		2	Minor inconsistencies
bookings	332754	311036	9016	311036	1	⚠ 16.9% bookings have driver_ids absent in drivers (ghost drivers)
offers	3130725	3115181	10616	311501	15544	⚠ ~21% offers to ghost drivers (4425)

Upstream Data Gap: Analysis reveals a significant discrepancy between transactional data and driver snapshots. Roughly 17% of bookings contain "ghost" driver_ids—IDs that appear in bookings but do not exist in the driver records.

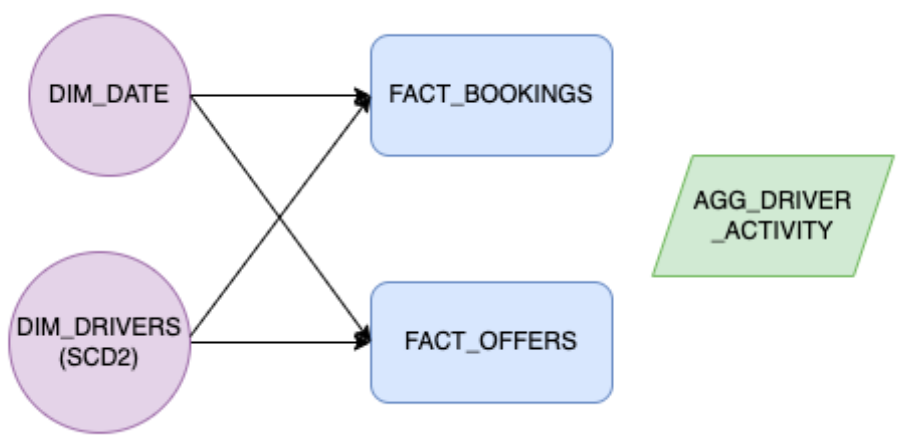
Source System Issue: This volume suggests a potential sync delay or an incomplete snapshot arrival from upstream sources. Ideally, these records would be blocked by **Referential Integrity (RI) checks**, but due to the high volume, they are currently permitted to avoid losing vital passenger demand data. Instead of exclusion, these records are flagged for transparency. This allows for accurate "Total Booking" counts while isolating the "ghost" records that would otherwise skew driver-specific performance metrics.

Recommended Action: Audit of the source systems and upstream pipeline is required to resolve the missing driver snapshots and restore full referential integrity.

6. Dimensional Modelling - Marts

Constellation schema with 2 fact tables and 2 dimensions

Scripts: dim_date.sql -> dim_drivers.sql -> fact_bookings.sql -> fact_offers.sql -> agg_driver_activity.sql



DIM_DATE (Query in dim_date.sql)

A conformed date dimension that ensures consistent time-based reporting across the data model. It provides calendar attributes (day, week, weekend, month, quarter, year) to support daily aggregation and trend analysis. Since we need to analyse only 14 days of data, the table also has dates fo**r 2021-June** alone.

DIM_DRIVERS (Query in dim_drivers.sql)

The source data is snapshot-based, and attributes such as country, receive_marketing_preference, or rating can change over time. Therefore, the table is modeled as a **Slowly Changing Dimension** (SCD) Type 2 to preserve historical changes.

Implementation Details

The table includes:

- start_eff_ts - start effective timestamp
- end_eff_ts - end effective timestamp
- is_active - flag to indicate current active record
- last_modified_ts -used to track the most recent change applied to the record

SCD Type 2 is implemented in two steps:

- Close the existing active record
  - Update end_eff_ts to source.raw_load_ts - 1 second
  - Set is_active = FALSE

- Insert a new record
  - start_eff_ts = source.raw_load_ts
  - end_eff_ts = '9999-12-31'::TIMESTAMP_TZ
  - is_active = TRUE

Note: This is the only mart-level table loaded incrementally, as it tracks attribute-level historical changes. The view **vw_dim_drivers** contains only active records (is_active = TRUE) and includes an additional derived field, **driver_type**, which classifies drivers as Veteran or Rookie based on their registration_date.

FACT_BOOKINGS & FACT_OFFERS (Query in fact_bookings.sql & fact_offers.sql)

Note: In a production environment, both fact tables would ideally be loaded incrementally to ensure scalability and cost efficiency. However, given the small dataset size and the scope of this task, both facts are implemented as truncate-and-load for simplicity and ease.

FACT_BOOKINGS

- One row per booking (booking-level grain).
- Represents the lowest level of granularity and is not pre-aggregated.

FACT_OFFERS

- One row per offer (driver-offer level grain).
- Multiple offers may exist for a single booking, as the same booking can be offered to multiple drivers.

Use of dim_driver_id in fact tables

Added dim_driver_id (surrogate key) in both fact_bookings and fact_offers for consistency with dimensional modeling best practices. Currently, a significant portion of driver records are missing in dim_driver, so the key does not cover all rows. Its impact on current analyses is limited, but having it ensures smooth integration once data quality improves and late-arriving drivers are resolved.

7. DATA PRODUCT - AGG_DRIVER_ACTIVITY (Query in agg_driver_activity.sql)

This table provides a **daily, driver-level** view of activity by combining bookings and offers, enabling monitoring of driver engagement and performance.

Grain: One row per driver per day. Includes all drivers active during the period, even if they had no offers or bookings on a given day.

KPIs included:

- **Offers**: total offers, accepted, cancelled, read, route distance
- **Bookings**: total bookings, completed trips, driver aborts, no driver found, total estimated revenue
- **Calculated ratios:** offer acceptance rate, booking completion rate, driver abort rate, average revenue per trip

Notes:

- Missing driver/booking records are flagged (is_missing_driver, is_missing_booking) to track data quality.
- Metrics are calculated independently at a daily level to account for advanced bookings and timing misalignment between offers and bookings.

- This is **not a cohort-based model** — bookings and offers are not tracked as linked journeys within a specific period. Instead, activity is measured in **absolute daily terms** based on event dates.

Optimizations Applied / Recommended for agg_driver_activity

To ensure fast access for analysts, the table is **pre-aggregated** at the driver-per-day level.
[Recommended]

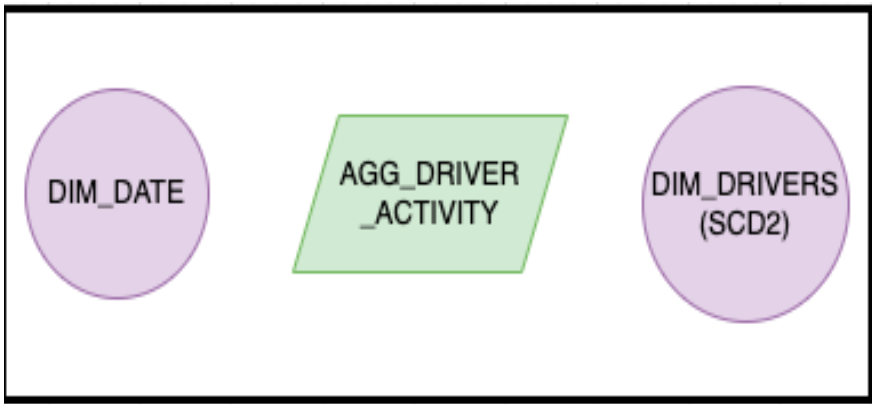
- At the storage level, c**lustering the table on activity_date_key and** (if needed)**driver_id**—the most active filter columns—can significantly improve query performance.
- Since I used Snowflake, using **larger warehouses** could result in faster query execution/ retrieval.

Optimisation - not just for faster access

- While building the query, SQL optimizations include filtering by date before joins, using **integer-based date keys** (activity_date_key) for efficient joins, and deduplicating without expensive **DISTINCT** operations.
- For ongoing usage, switching from **full rebuilds to incremental daily loads** would reduce processing time and resource consumption.

8. ANALYSES (Query in analyses.sql + EDA.ipynb)

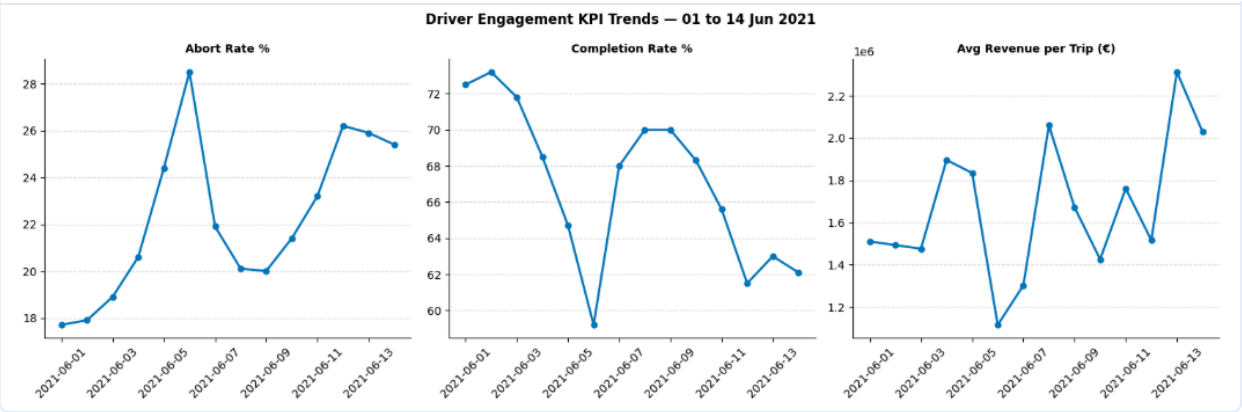
Analyses Approach:
Using the **agg_driver_activity** table as the base, views were created to slice KPIs by dimensions such as weekday vs. weekend, driver type (Veteran vs. Rookie), and time (analyses.sql).



Basic exploratory analyses were conducted in SQL and Python (EDA.ipynb) to track metrics over the period. Key metrics monitored include:

- **Driver Abort Rate** – proportion of bookings aborted by the driver
- **Completion Rate** – proportion of successful bookings completed
- **Average Estimated Revenue per Trip** – average of estimated revenue per completed booking

Analysis 1 — KPI Trends Over Time (01–14 Jun 2021)



Three core KPIs were tracked daily across the 14-day period: driver abort rate, booking completion rate, and average revenue per completed trip (estimated).

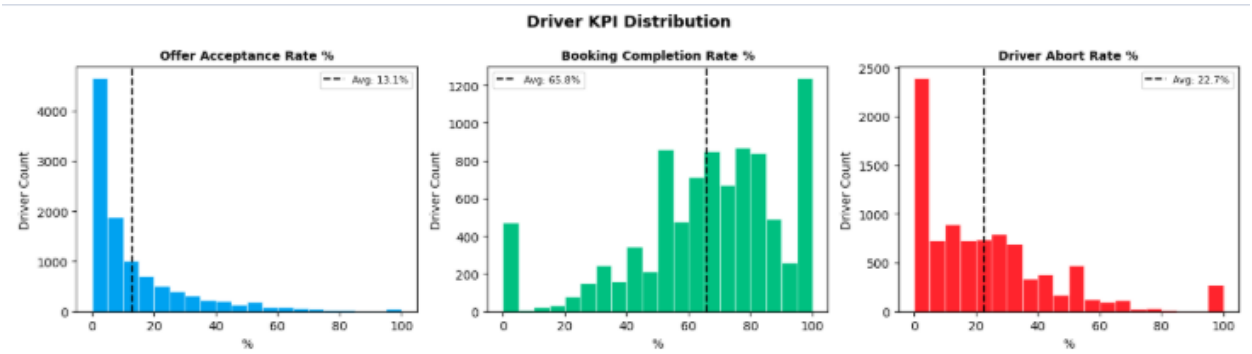
Abort rate rose from 18% on Jun 1 to 26% by Jun 13, with a sharp spike to **29% on Jun 5**. Completion rate moved in the opposite direction — opening at 72% and declining to 62% by end

of period, with a notable crash to **57% on Jun 7**, two days after the abort spike. This lag is expected — aborts on one day reduce completions on subsequent days. Revenue per trip was volatile but showed an upward trend toward the end of the period, suggesting fewer but higher-value trips were completing as the period progressed.

The relationship between these three metrics points to a **driver behaviour problem** rather than a demand problem. **Rising aborts** are directly suppressing completion rate, which in turn impacts revenue. Every abort represents a blocked booking, a delayed passenger, and a lost fare for that trip slot.

Recommendation: Introduce an **abort rate threshold** — drivers exceeding 30% abort rate over a 7-day window should be deprioritised in offer routing, giving reliable drivers first access to new bookings. Success should be measured by completion rate recovering toward 70%+ within two weeks of intervention.

Analysis 2 — Driver KPI Distribution



A static snapshot of all 10,616 drivers reveals three structural patterns in the driver base:

Acceptance rate is heavily right-skewed — majority of drivers have acceptance rates between 0–10%, with a platform average of just 13.1%. This indicates a large disengaged majority who receive offers but rarely act on them.

Completion rate is left-skewed — most drivers who do receive bookings complete them at a high rate (70–100% range), with a platform average of 65.8%. Once a driver commits, they generally follow through.

Abort rate is right-skewed with a long tail — while most drivers cluster near 0–10% abort rate, a distinct group sits in the 40–60% range. These are drivers who are accepting bookings and then cancelling them at high frequency.

Low acceptance (13%) + high completion (66%) + elevated abort average (23%) implies— there is a large disengaged group who rarely engage with offers, and a smaller but highly disruptive minority who accept and then abort. 2,775 drivers (approximately 30% of the active base) fall into the high abort segment, and this group generates disproportionately fewer completed trips and lower revenue per trip compared to reliable drivers.

Recommendation: These two groups require different strategies. The disengaged majority may respond to re-engagement campaigns — particularly those who have opted into **marketing communications**. The high-abort minority requires to be dealt with penalties to bring down the abort rate.

9. Next Steps

- **Parameterize Hard-Coded Values:**
  Replace fixed date filters and thresholds with configurable variables to improve flexibility and reusability of queries.

- **Implement dbt:**
  Migrate transformations to dbt to enable modular modeling, version control, testing (e.g., uniqueness, not-null checks), and better documentation of the data pipeline.

- **Build a Streamlit Dashboard:**
  Develop a lightweight interactive dashboard to visualize driver engagement KPIs (acceptance rate, completion rate, abort rate, revenue trends) and enable dynamic filtering by date, driver type, or other dimensions.